

# RBF-FD Solver: Code Structure and Workflow

Dennis Corraliza

6/21/2026

## Abstract

This note describes the structure of the RBF-FD (Radial Basis Function Finite Difference) PDE solver implemented in `assembly.py`, `domain.py`, `rbf.py`, `boundary.py`, `stencils.py`, and `geometry.py`. It walks through the pipeline from node generation to solved nodal values, module by module, with just enough mathematics to explain *why* each step exists. For the full derivation of the underlying RBF-FD numerics, see the companion theory note.

## Contents

|           |                                                                   |          |
|-----------|-------------------------------------------------------------------|----------|
| <b>1</b>  | <b>What the solver computes</b>                                   | <b>1</b> |
| <b>2</b>  | <b>Module map</b>                                                 | <b>2</b> |
| <b>3</b>  | <b>Pipeline diagram</b>                                           | <b>3</b> |
| <b>4</b>  | <b>Step 1: Node generation (<code>geometry.py</code>)</b>         | <b>4</b> |
| <b>5</b>  | <b>Step 2: Stencils (<code>stencils.py</code>)</b>                | <b>4</b> |
| <b>6</b>  | <b>Step 3: The domain context (<code>domain.py</code>)</b>        | <b>4</b> |
| <b>7</b>  | <b>Step 4: Choosing a kernel (<code>rbf.py</code>)</b>            | <b>4</b> |
| <b>8</b>  | <b>Step 5: Choosing a domain shape (<code>boundary.py</code>)</b> | <b>5</b> |
| <b>9</b>  | <b>Steps 6–8: Assembly (<code>assembly.py</code>)</b>             | <b>5</b> |
| 9.1       | Local weight solves . . . . .                                     | 5        |
| 9.2       | Global assembly . . . . .                                         | 5        |
| 9.3       | Pure-Neumann anchoring . . . . .                                  | 5        |
| <b>10</b> | <b>Step 9: Solving (<code>assembly.rbf_fd_solve</code>)</b>       | <b>6</b> |
| <b>11</b> | <b>Putting it together: the top-level driver</b>                  | <b>6</b> |
| <b>12</b> | <b>Validating the solver: manufactured solutions</b>              | <b>6</b> |

## 1 What the solver computes

Given a domain  $\Omega$  (a square or a disk), a diffusion tensor  $A$ , a source term  $f$ , and boundary data, the solver produces an approximate solution  $u$  to

$$-\nabla \cdot (A \nabla u) = f \quad \text{in } \Omega,$$

with Dirichlet conditions ( $u$  prescribed) and/or Neumann conditions ( $\partial u / \partial n$  prescribed) on different parts of the boundary. It does this by:

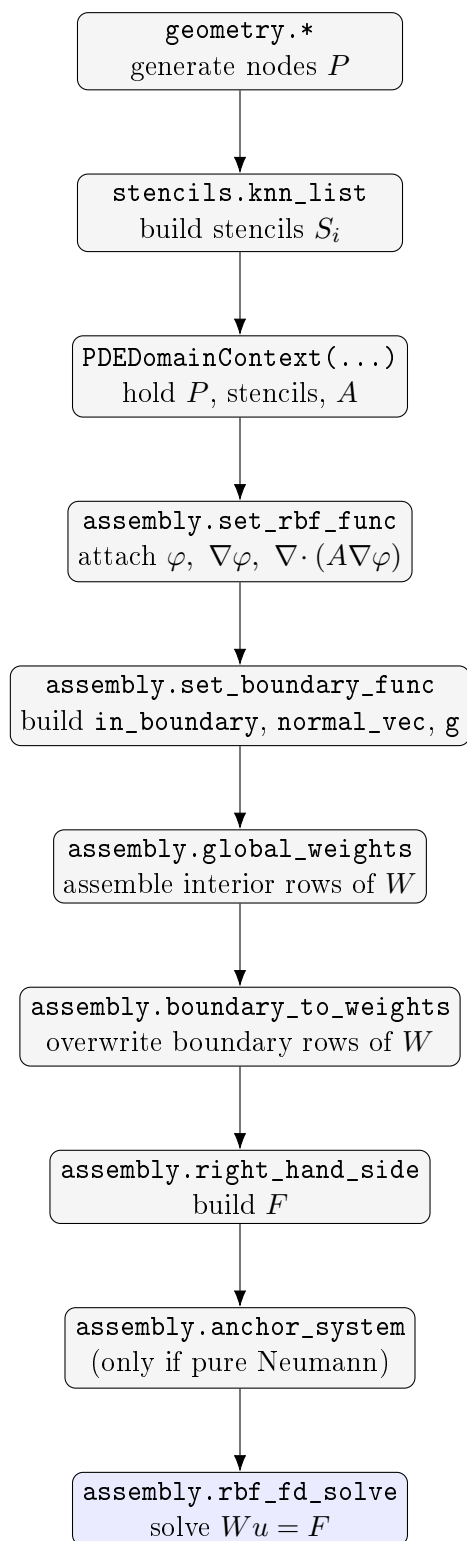
1. scattering nodes over  $\Omega$ ,
2. building, for each node, a small local stencil of nearby nodes,
3. fitting local differentiation weights at each node so that a weighted sum of neighboring values approximates the PDE operator there,
4. assembling all the local weights into one big sparse-like (here, dense) matrix  $W$ ,
5. overwriting the boundary rows of  $W$  to enforce the boundary conditions,
6. solving the resulting linear system  $Wu = F$ .

The rest of this note walks through which file and function does each step.

## 2 Module map

| Module                   | Responsibility                                                                                                                     |
|--------------------------|------------------------------------------------------------------------------------------------------------------------------------|
| <code>geometry.py</code> | Generates node coordinates for a given domain shape.                                                                               |
| <code>stencils.py</code> | Builds nearest-neighbor stencils for every node.                                                                                   |
| <code>rbf.py</code>      | Kernel functions, their gradients, and the (anisotropic) Laplacian applied to each kernel; auxiliary grid generation.              |
| <code>boundary.py</code> | Classifies nodes as interior/Dirichlet/Neumann; supplies boundary values and outward normals.                                      |
| <code>domain.py</code>   | <code>PDEDomainContext</code> : a plain data container holding everything (nodes, stencils, kernel choice, $A$ , $W$ , $F$ , ...). |
| <code>assembly.py</code> | The actual RBF-FD machinery: local weight solves, global assembly, boundary imposition, and the top-level driver.                  |

### 3 Pipeline diagram



All of these steps (except node generation and the final solve) are chained together for you by `assembly.rbf_fd_system(...)`, which is the function most user code should actually call.

## 4 Step 1: Node generation (geometry.py)

Four node-generation strategies are provided. They all return a plain array  $P \in \mathbb{R}^{N \times 2}$  of node coordinates; nothing downstream cares how  $P$  was produced.

- `uniform_square(L, Nx, Ny)` – regular grid, boundary and interior points mixed together.
- `uniform_int_square(L, Nx_int, Ny_int, Nb)` – regular interior grid, plus  $N_b$  points per edge on the boundary, with corners de-duplicated. Returns `(P, num_interior)`.
- `cheby_square(L, Nx, Ny)` – Chebyshev–Gauss–Lobatto points, clustered near the edges (standard for spectral methods).
- `circular_geometry(R, Nx, Ny, num_bound)` – grid points filtered to the interior of a disk, plus points placed exactly on the circle.

```
P, num_int = geometry.uniform_int_square(L=1.0, Nx_int=40, Ny_int=40, Nb=80)
```

## 5 Step 2: Stencils (stencils.py)

For each node  $\mathbf{x}_i$ , `stencils.knn_list(P, k_s)` finds the *num\_stencil\_nodes* nearest neighbors of  $\mathbf{x}_i$  (including itself) by Euclidean distance, and returns a list `stencils[i]` of their indices in  $P$ . This list is the only thing that determines which other nodal values can contribute to the discrete operator at  $\mathbf{x}_i$ .

## 6 Step 3: The domain context (domain.py)

`PDEDomainContext` is a plain container, not a solver — it just holds state so functions in `assembly.py` don't need ten positional arguments each. It is built once, then configured incrementally with several `set_*` calls:

```
context = domain.PDEDomainContext(P, stencils, A)
context.set_phi(...)
context.set_grad_phi(...)
context.set_laplacian_phi(...)
context.set_augmentation(True) # optional
context.set_centers(num_rings) # optional: enables least-squares weights
```

By the end of the pipeline it also holds the assembled weight matrix (`context.W`) and right-hand side (`context.F`), set via `set_W` / `set_rhs`.

## 7 Step 4: Choosing a kernel (rbf.py)

Two radial kernels are available, selected by the string `basis` passed to `assembly.set_rbf_func`:

| basis      | Kernel $\varphi(\mathbf{p})$           | Functions                                                                                              |
|------------|----------------------------------------|--------------------------------------------------------------------------------------------------------|
| 'cubic'    | $\ \mathbf{p}\ ^3$                     | <code>phi_cubic</code> , <code>grad_phi_cubic</code> ,<br><code>anisotropic_diffusion_phi_cubic</code> |
| 'gaussian' | $\exp(-(\varepsilon\ \mathbf{p}\ )^2)$ | <code>phi_gauss</code> , <code>grad_phi_gauss</code> ,<br><code>anisotropic_diffusion_phi_gauss</code> |

The `anisotropic_diffusion_phi_*` functions evaluate  $\nabla \cdot (A \nabla \varphi)$  in closed form (not numerically); this is what makes the RBF-FD weights exact for the kernel rather than an approximation of one. If `augmentation=True`, the linear polynomial basis  $\{1, x, y\}$  (`poly_basis`, `grad_poly`, `anisotropic_diffusion_poly`) is also added, so that the local fit reproduces constants and linear functions exactly.

## 8 Step 5: Choosing a domain shape (`boundary.py`)

`assembly.set_boundary_func` dispatches on the string `shape` (`'square'` or `'circle'`) to build three closures:

|                | square                          | circle                            |
|----------------|---------------------------------|-----------------------------------|
| classify node  | <code>in_square_boundary</code> | <code>in_circular_boundary</code> |
| outward normal | <code>square_normal</code>      | <code>disk_normal</code>          |
| boundary value | <code>square_boundary</code>    | <code>circular_boundary</code>    |

For the square, boundary data is supplied as a list of four one-variable functions [`g1`, `g2`, `g3`, `g4`] for the bottom, right, top, and left edges respectively (ordered counterclockwise starting at  $y = 0$ ); for the disk, a single function `g(p)` covers the whole boundary circle.

## 9 Steps 6–8: Assembly (`assembly.py`)

### 9.1 Local weight solves

For each node, `local_weights_solve` (or, if `context.center_rings` is set, `local_weights_ls`) computes the row of differentiation weights. `local_grad_solve/local_grad_ls` do the same for the gradient, needed only at Neumann boundary nodes. These four functions are the computational core of the method; everything else in `assembly.py` is bookkeeping around them.

### 9.2 Global assembly

```
W = assembly.global_weights(context, in_boundary, normal_vec)
assembly.boundary_to_weights(W, context, in_boundary, normal_vec) # in-place
F = assembly.right_hand_side(context, f, g, in_boundary)
```

`global_weights` loops over every node and scatters its local weights into a dense  $N \times N$  matrix  $W$ . `boundary_to_weights` then overwrites, in place, the rows of  $W$  corresponding to Dirichlet nodes (identity row) and Neumann nodes (directional-derivative row). `right_hand_side` fills  $F$  with  $f$  at interior nodes and the boundary data at boundary nodes.

### 9.3 Pure-Neumann anchoring

If every boundary node is Neumann (checked by `is_pure_neumann`),  $W$  is singular (the solution is only determined up to a constant), so `anchor_system(W, F, method)` regularizes it using one of three strategies (`'mean'`, `'pin'`, or `'project'`) before solving.

## 10 Step 9: Solving (assembly.rbf\_fd\_solve)

```
u = assembly.rbf_fd_solve(context.W, context.F)
```

A thin wrapper around `numpy.linalg.solve`: nothing RBF-FD-specific happens here, since all of the discretization work was already baked into  $W$  and  $F$ .

## 11 Putting it together: the top-level driver

In practice, a user only needs to call:

```
context = assembly.rbf_fd_system(  
    f, g_bound, btype, P,  
    basis='cubic', shape='square', L=1.0,  
    num_stencil_nodes=25, num_rings=None, # num_rings=None -> direct solve; num_rings=int  
        -> least squares  
    augmentation=True, A=my_tensor  
)  
u = assembly.rbf_fd_solve(context.W, context.F)
```

`rbf_fd_system` runs steps 3 through 8 end-to-end (printing a short progress message after each stage) and returns a fully populated `PDEDomainContext`, ready to be handed to `rbf_fd_solve`.

## 12 Validating the solver: manufactured solutions

To check that the solver is actually correct, test problems are built by picking a closed-form  $u_{\text{exact}}$ , then differentiating it analytically to obtain a matching  $f$  and boundary data — this is the *method of manufactured solutions*. Each such test problem is a small Python function (e.g. `example_3`) returning the tuple `(f, g, btype, u_exact)`, ready to be passed straight into `rbf_fd_system`. Once solved, `report_and_graph(context, u_exact)` compares the numerical solution against  $u_{\text{exact}}$  at every node and reports:

- the condition number of  $W$ ,
- the maximum pointwise error and a discrete  $L^2$  error,
- contour and 3D surface plots of the approximate solution, the exact solution, and the pointwise error.

This is the standard way to sanity-check a new kernel, stencil size, or boundary condition setup before trusting the solver on a problem without a known exact solution.